Secure Over-the-Air Updating

Andrew Rucks
Pennsylvania College of Technology
Williamsport, PA
asr22@pct.edu

***Abstract*—The number of interconnected devices in the world has increased greatly: smartphones, wireless earbuds, smart thermostats, doorbell cameras, embedded industrial systems, and other gadgets are commonplace. Even machines that have previously been considered "offline"—home appliances, televisions, and cars—have joined in on this trend. These interconnected gadgets run on firmware and software that inevitably require updating by the manufacturer, whether that be to fix bugs or patch vulnerabilities (needs driven by both the market and regulations); this is where the concept of secure over-the-air (OTA) updating is useful. Secure OTA updates are updates to firmware/software that are delivered over the internet (wireless in particular) with confidentiality and integrity and can be automatically rolled back in the event of failure. The goal of this paper is to explain, based on research of published articles and documentation, how OTA updates generally work and how they can be securely delivered to recipient devices. This explanation will touch on network infrastructure, server-side update delivery software, host-side software and firmware, encryption, digital signing and verification, and more. Inevitably, with the discussion of firmware, circuit board mechanisms like hardware security modules (HSM) are discussed. Lastly, different models and frameworks for the implementation of secure OTA are compared.**

I. Introduction

ANY devices, machines, or gadgets that connect to the internet will have firmware and software on board. These firmware and software components will inevitably experience bugs and glitches, security issues and vulnerabilities, or the market-driven desire for new features. Moreover, regulations like the European Union's Cyber Resilience Act, that requires protecting the integrity of code and data on devices, and ensuring "that vulnerabilities can be addressed through security updates," have been enacted.[1] If firmware or software has already been deployed, perhaps in thousands to millions of devices around the world, how can manufacturers update myriads of devices securely and remotely to meet those inevitable needs? The contemporary answer is secure over-the-air (OTA) updating.

The kinds of devices and machines that secure OTA is relevant for (especially as more and more products with internet connectivity are distributed) include smartphones, wireless earbuds, smart refrigerators and other appliances, smart thermostats, doorbell cameras, televisions, floor-vacuuming robots, industrial systems and sensors, and cars (not just self-driving ones—many regular cars have internet connections now), and more. Broadly, these can be categorized as internet-of-things (IoT) devices or cyber-physical-systems (CPS).[2]

According to a whitepaper by Texas Instruments, OTA updates "enable remote patching of bugs or security flaws, rather than having expensive service technicians or inexperienced users perform the updates in-person."[3] Secure OTA updates can deliver anything from small patches to major firmware upgrades, but what all secure OTA updates have in common is they are delivered to the end device via Wi-Fi, cellular, Bluetooth, or wired internet, and, they are delivered in a secure manner that complies with the principles of integrity, availability, and confidentiality. The precarity of delivering major updates to IoT/CPS devices over the internet adds a third requirement: automatic rollback, recovery, or "reversion" if the update fails.[3]

There are several published frameworks and software tools for implementing secure OTA, including Uptane, The Update Framework (TUF), ISO 24089, STRIDE, and commercial solutions. These will be compared and discussed in this paper's final section. In the next section, the basic theory of OTA updates is summarized.

II. Over-the-Air Updates

There are three puzzle pieces in an OTA update scenario. First, is the end user device or client (e.g. a smart thermostat or connected vehicle). This device continually checks for updates by querying the manufacturer's designated update servers, or in other words, runs an update daemon. It will then receive and install the updates. Next, is an update delivery server/service. This server is responsible for transmitting the update or patch files to the end user device. Lastly, there are verification servers or services, that can authenticate connecting devices, encrypt transmitted update files, and provide file integrity verification and code signing information.[4] These will be examined in the next section as part of the discussion of security.

III. Securing Over-the-Air Updates

Here is an example of a secure OTA update system: 1) in cloud infrastructure, the OTA update image is created, digitally signed, published for a group of IoT/CPS devices, and

For IAS-490: Advanced Topics in Information Assurance & Security. Submitted March 15, 2026.

eventually uploaded to said devices; meanwhile, 2) each IoT/CPS device checks for updates, downloads the update image, authenticates it, and boots into the new image; finally, 3) the new image is committed to unless it fails, in which case the old image is reverted.[3] Other security mechanisms like encryption are used between the cloud infrastructure and the end device. These mechanisms combine to protect against common attacks against update systems, like spoofing (the attacker impersonates a supposedly friendly host), man-in-the-middle (the attacker intercepts, eavesdrops, or manipulates communications between hosts), distributed denial-of-service (the attacker overwhelms a system from multiple points), and duplicate update (attackers resend an update message that has already been sent) attacks.[5]

A. Integrity

Ensuring the integrity of updates is arguably the driving need for OTA update systems. What are major threats to the integrity of files sent "over the air"? Accidental corruption due to connection issues and purposeful tampering by malicious actors. Code signing is an excellent solution to counter these threats. In a basic code signing scenario, there are three roles: the developer, the signer, and the verifier.[6] The developer is the organization that writes and releases the code or update file (hopefully, the same organization that sells the IoT/CPS device) After developing and testing the code, it is submitted to the signer, a role that can be fulfilled by the same organization, or by a trusted third party. Code signing relies simultaneously on the principles of asymmetric encryption, in which content encrypted by a "public" key can only be decrypted by a corresponding and secret "private" key, and vice-versa, and hashing, which calculates a unique "digest" of the content that can instantly reveal if the content has been modified.[7] The signer is responsible for generating private and public keys, registering the public key with a certificate authority (CA), and hashing the code. The third code signing role, the verifier, lies with the IoT/CPS device itself. After downloading the update file from the developer/manufacturer, the end device generates a hash of the file, then verifies its legitimacy by comparing it to the hash provided by the signer and CA.[6] If the code signing infrastructure is secure, and the hashes match, then integrity has been achieved, and the device will proceed with the update.

B. Availability

The next concern of secure OTA updating is availability. In this case, availability refers to the ability of update delivery servers and code signing infrastructures to transmit the necessary content to CPS/IoT devices in a timely manner. Ensuring OTA availability raises several questions: are there update servers in multiple geographic locations? Does the connection to the wide area network (WAN) have sufficient bandwidth for updating thousands of devices simultaneously? What kind of connection are the end devices using to download the update? —Wi-Fi? Bluetooth? Cellular? What if the device moves between cell towers or access points during an update? What networking protocols will be used? Different cases call for different answers. Besides the update delivery aspect, the availability of the IoT/CPS device itself is a concern. Downloading and installing updates requires a drain of computing resources that affect the availability of IoT/CPS devices. "For applications such as remote sensors that report data only periodically, it may be acceptable to cease normal operation and immediately boot the OTA update image. But an OTA update must not stop a robotic cleaner or smart coffee machine in the midst of a job."[3] After the update is installed another availability hurdle remains: does the new software version even work? Granted, proper testing during development should make post-installation issues rare. But what if, for example, there is a power outage during installation that corrupts a part of the new software? A secure OTA system would be prepared for this contingency by firstly not overwriting the old firmware or software components, and secondly, running a test with the initial boot of the new version. If that test fails or the IoT/CPS device fails to even boot, it will return to the old version, and at least still have availability.

C. Confidentiality and HSMs

Keeping unauthorized eyes from accessing sensitive update files is relatively less of a concern for OTA updating, but definitely worth consideration One part of ensuring confidentiality is encrypting data in transit, which can be as simple as using transport-layer security (TLS) protocols like HTTPS to conceal the update files as they are transmitted to the end device.[5] The rationale for this is to prevent attackers from detecting that an IoT/CPS device is updating or sniffing sensitive data that may be sent during the update, which may provide an opening to attack, and to prevent (or at least hinder) attackers from copying the update code. Attackers can perform static analysis on the update code to expose vulnerabilities that may have been missed during development. The update code sent to end devices is hopefully compiled (from source code that is even more important to protect on the development side), and thus harder to analyze, but it is still a risk that must be considered. A common solution for protecting update code on end devices is to add a hardware security module (HSM) to every device's circuit board. HSMs are "specialized devices that perform cryptographic operations and store private-public key pairs and their associated secret values"; they can generate and manage keys, perform encryption, decryption, and hashing.[8] Not only can HSMs assist with the integrity and availability aspects of OTA update security, but they can also be used to encrypt IoT/CPS devices' update files in storage.

IV. Frameworks and Solutions

In the previous section, the central functions of secure OTA updating have been described. Now, real-world models, tools, and frameworks for implementing secure OTA can be discussed and compared. It must be noted that OTA updating is not one-size-fits-all, with variability in the kinds of IoT/CPS devices being updated, the number of those devices worldwide, the chosen mode of update delivery, and the manufacturer's risk appetite. Different situations require different solutions.

A. Software Frameworks and Solutions

The Update Framework (TUF) is an open-source project aimed at securing update delivery systems and software repositories. The framework can be included in a new update system or added to an existing one, and it has been implemented in the Python, Go, JavaScript, and Rust programming languages. TUF security relies on a few principles: *trust*, which means assuming downloaded files are "without malicious designs," *compromise-resilience*, which entails fast and secure key replacement and revocation that maintains security even in the event of private key disclosure, *integrity*, of both downloaded files and the repositories from which they originate, and *freshness*, which means being proactive about downloading new software versions.[9] TUF is designed for typical software updates rather than secure OTA updates of IoT/CPS devices, but it can be used as a starting point for specialized frameworks.

Thistle and Mender are two OTA solutions that are intended for commercial use. They both support updating Linux devices, Zephyr real-time operating system (RTOS) devices, and other embedded systems, and they operate by running a client program on each host device that communicates with a central update server.[10, 11] Alternatively, a manufacturer may build devices using a company like Particle, that offers an "IoT stack," including the specialized circuit boards, base operating system, and OTA update infrastructure.[12] While commercial solutions are (relatively) simple to implement and convenient to use, they cost money and cannot fit all scenarios, especially if the IoT/CPS device manufacturer uses custom firmware and hardware.

B. Automobile Updating

Uptane is a software update framework for automobiles, which is necessary because of the unique nature of updating vehicles: they have heightened safety concerns and multiple different computers in a single vehicle. It begins with the code signing abilities of the previously mentioned TUF, with additional modifications necessary to meet the challenges of securely updating electronic control units (ECUs). How Uptane works is relatively straightforward: To start with, the repository (update delivery server) provides a *signed* metadata file, that includes a hash of the new firmware version, alongside the update files themselves. Next, the repository transmits the update files and metadata as a bundle to a primary computer on the vehicle, using TLS or another protected channel for confidentiality. After receiving the bundle, the primary computer distributes the updates files to the intended secondary computers/ECUs, and interestingly, *broadcasts* the metadata to all ECUs simultaneously. Each ECU validates the update files using the associated metadata and then installs them onto a different storage section than the last version, to allow for emergency reversion. Additionally, Uptane calls for the primary computer to hold a manifest of firmware version numbers from each ECU, and for each ECU to periodically synchronize their clocks with an external time server via the primary computer's OTA connection.[13]

The International Organization for Standardization has published standards for securely updating vehicles: ISO 24089. It describes multiple topics including the "implementation of organizational level processes for software update engineering," the "assembly of software update packages," and the "preparation and execution of software update campaigns using functions in the vehicle and infrastructure," even though it doesn't prescribe "specific technologies or solutions" to use.[14]

STRIDE is a scheme for updating vehicles from the cloud that, besides focusing on security, specifically tackles the problem of delivering updates to potentially millions of vehicles at the same time. The authors propose using ciphertext-policy attribute-based encryption, which essentially bakes access control into a file itself rather than relying on security provided by the server that hosts it. Using this, vehicle manufacturers can host update files on a multitude of cloud servers, with varying degrees of security, globally, thus distributing the heavy network load.[15] As the list of vehicles and devices that require secure OTA updates continues to lengthen, more techniques like STRIDE will likewise be experimented with and refined.

REFERENCES

[1] European Union, Parliament, "Regulation (EU) 2024/2847 of the European Parliament and of the Council," *EUR-Lex*, Oct. 23, 2024. data.europa.eu/eli/reg/2024/2847/2024-11-20.

[2] M. Villegas, C. Orellana, and H. Astudillo, "A study of over-the-air (OTA) update systems for CPS and IoT operating systems," *ESCA '19: Proc. of the 13th Euro. Con. on Software Arch.*, volume 2, pp. 269–272, 2019, doi: 10.1145/3344948.3344972.

[3] N. Lethaby, "A more secure and reliable OTA update architecture for IoT devices," Texas Instruments, 2018. www.ti.com/lit/wp/sway021/sway021.pdf.

[4] "Guide: Secure over-the-air (OTA) updates," Secure-by-Design. www.securebydesignhandbook.com/docs/implementation/build-phase/ota-updates (accessed Mar. 15, 2026).

[5] Y. Shin and S. Jeon, "MQTree: Secure OTA protocol using MQTT and MerkleTree," *Sensors*, volume 24, no. 5, 2024, doi: 10.3390/s24051447.

[6] D. Cooper et. al., "Security considerations for code signing," NIST, 2018. csrc.nist.rip/external/nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.01262018.pdf

[7] R. Sander. "What is a code signing certificate? How does it work?" IEEE Computer Society. www.computer.org/publications/tech-news/trends/what-is-a-code-signing-certificate (accessed Feb 24, 2026).

[8] M. Sommerhalder, "Hardware security module," in *Trends in Data Protection and Encryption Technologies*, 2023, pp. 83–87. doi: 10.1007/978-3-031-33386-6_16.

[9] TUF Authors, "Security," *TUF*, Oct. 12, 2024. theupdateframework.io/docs/security/ (accessed Mar. 15, 2026).

[10] Thistle Authors, "Thistle OTA update overview," *Thistle Docs*. docs.thistle.tech/update/overview (accessed Mar. 15, 2026).

[11] Northern.tech Authors, "How Mender works," *Mender*. mender.io/engineers/how-mender-works (accessed Mar. 15, 2026).

[12] Particle Authors, "OTA firmware updates," *Particle Docs*. docs.particle.io/getting-started/cloud/ota-updates (accessed Mar 13, 2026).

[13] T. K. Kuppusamy et. al., "Uptane: securing software updates for automobiles," Uptane, 2016. uptane.org/assets/files/kuppusamy_escar_16-320b8a87bf6d0dd9ff881fb989320ab1.pdf.

[14] Technical Committee 22 Authors, "ISO 24089, Road vehicles - Software update engineering: Introduction," *ISO*, Feb. 2023. www.iso.org/obp/ui/en/#iso:std:iso:24089:ed-1:v1:en

[15] A. Ghosal, S. Halder and M. Conti, "STRIDE: Scalable and Secure Over-The-Air Software Update Scheme for Autonomous Vehicles," *2020 IEEE International Conference on Communications*, 2020, pp. 1-6, doi: 10.1109/ICC40277.2020.9148649.